Snakes and Ladders

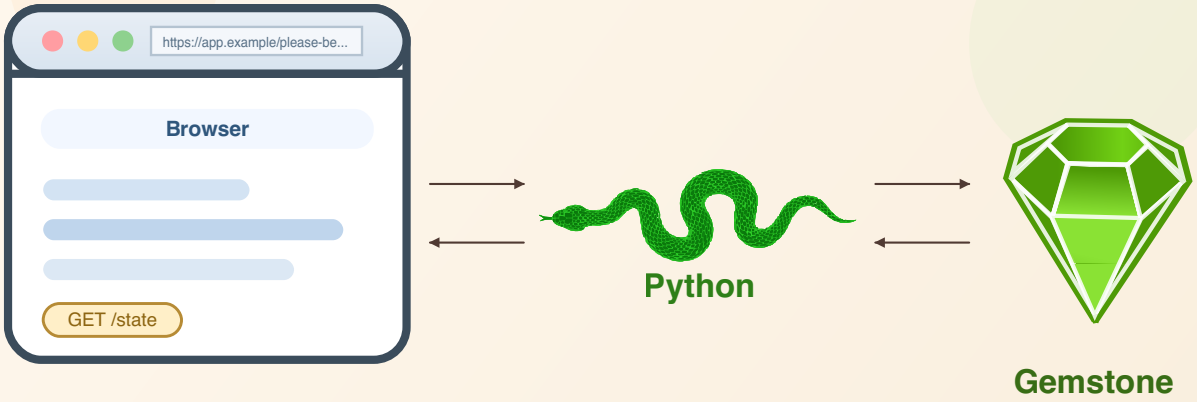


Table of Contents

A web browser asks polite questions.
Python does the clever legwork.
The Gemstone remembers everything and forgives nothing.

```
pip install gemstone-py
```

Contents

A Funny but Thorough Introduction to gemstone-py

Structure

The book is split into seven parts:

1. Part I: Why gemstone-py Exists
2. Part II: Sessions and Transactions
3. Part III: PersistentRoot and Friends
4. Part IV: Queries, Stores, and Logs
5. Part V: Web Apps and Request Lifecycles
6. Part VI: Concurrency, Conflicts, and Retries
7. Part VII: Benchmarks, Releases, and Operator Survival

Tone

This book follows a simple rule:

> If a topic is important, explain it clearly. If it is dangerous, explain it clearly and make the joke sharper.

That means:

- transaction policy sections are serious
- commit conflict sections are serious but slightly theatrical
- queue jokes are regrettably inevitable

Suggested Reading Paths

New user

Read Parts I, II, and III first.

Web-focused user

Read Parts I, II, and V first.

Async or typed-API user

Read Parts I, II, III, and V first, then keep the User Manual open for the focused `AsyncSession`, `TypedOop[T]`, and managed-lifetime examples.

Operations-focused user

Read Parts II, VI, and VII first.

Person who enjoys complete narratives

Read the whole thing in order and accept that you are now the office historian.

Part I: Why gemstone-py Exists

The Elevator Pitch

If somebody corners you near a whiteboard and asks what this package is, say:

> "`gemstone-py` is a Python package that talks directly to GemStone Smalltalk, keeps transaction intent explicit, and provides persistence, query, logging, concurrency, async, typed-access, web, benchmark, native-extension, and release tooling around that core."

That is the sober pitch.

The unsober pitch is:

> "It is what happens when a Python package decides that vague database magic is overrated and repository-backed honesty is more interesting."

Both are true. One is more likely to get you invited back to meetings.

The Problem Behind the Package

Talking to GemStone from Python sounds simple right up until the moment you need to do it in a way that:

- works repeatedly
- survives packaging
- survives release automation
- behaves correctly under web request lifecycles
- fits async Python applications without sharing one GCI session across threads

- gives typed and managed handles when raw OOP integers are too vague
- handles commit conflicts like an adult
- and still has examples clear enough for a new user

Without a real package, teams drift toward a recognizable archeological layer:

- one script that loads `libgcirpc` directly
- one script that mostly works but depends on the author's shell history
- one Flask integration that silently commits on a handled `500`
- one benchmark nobody trusts
- and one README that starts with confidence and ends with folklore

The point of `gemstone-py` is to avoid that outcome.

The Package's Temperament

`gemstone-py` is opinionated in a specific way:

- it prefers explicit transaction policy
- it prefers tested installable artifacts over checkout-only convenience
- it prefers maintained examples over nostalgic stubs
- it prefers benchmark evidence over volume
- it prefers boring release automation over brave improvisation

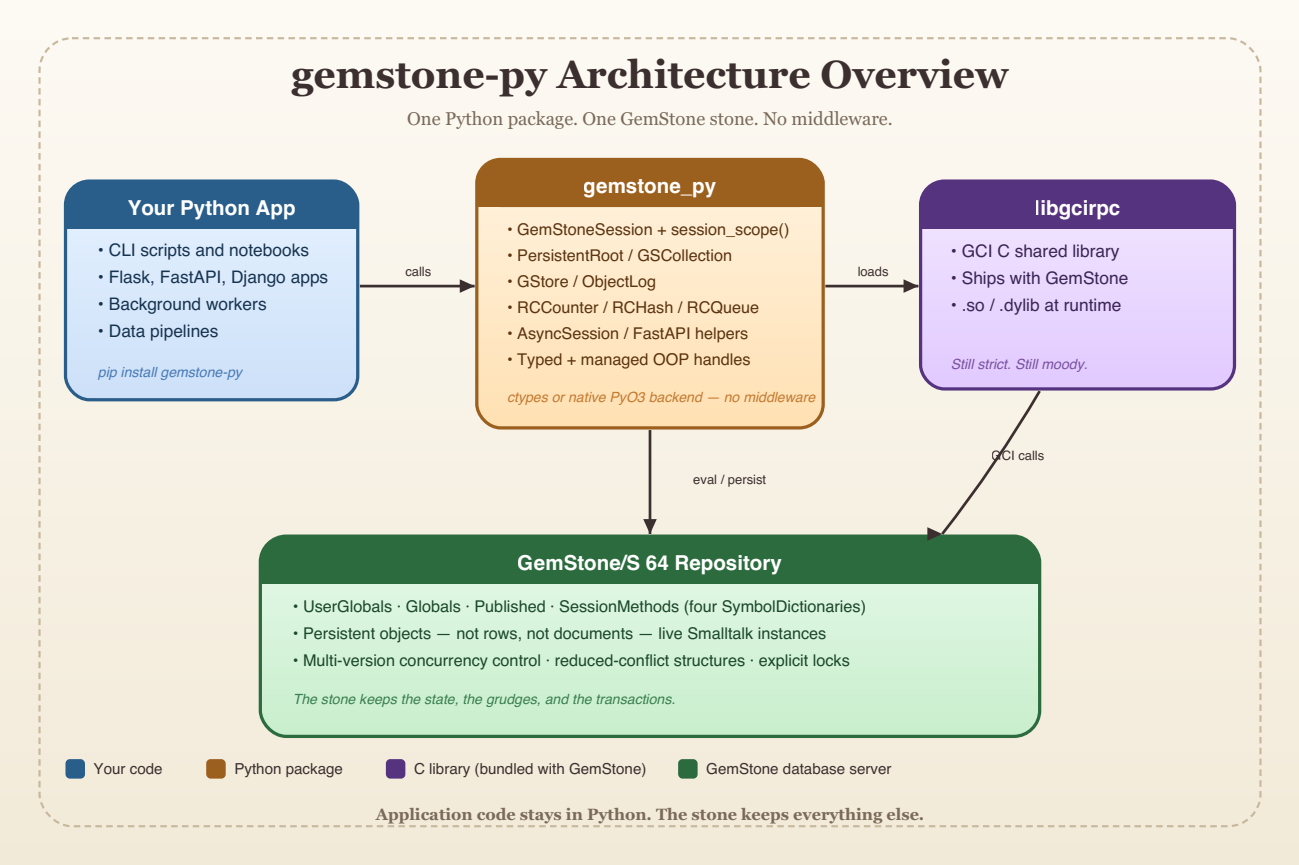
This is good news for users and mildly disappointing news for chaos.

You will notice the package is not trying to become:

- a mystical ORM
- a general-purpose object database tutorial
- a framework that pretends GemStone no longer exists

It remains a Python package for people who know that the stone is real and the repository is not a metaphor.

The Stack in One Picture



Read the diagram from left to right:

1. your Python code
2. the `gemstone_py` layer
3. `libgcirpc`
4. the GemStone repository

That picture matters because it prevents two common misunderstandings.

The first misunderstanding is that Python somehow "becomes" GemStone. It does not. Python stays Python.

The second misunderstanding is that all persistence helpers are merely local objects with a distant hobby. They are not. The data structures they front are real repository state.

Why the Package Is Better Than a Pile of Glue

A pile of glue can absolutely talk to a stone.

It can also:

- leak transaction assumptions

- hide login behaviour in global state
- ship with unrepeatable environment instructions
- and become terrifying the first time a release workflow must publish to PyPI

`gemstone-py` is better than that because it has already done the boring work:

- packaging
- typing
- unit tests
- live tests
- benchmark reports
- benchmark comparison
- baseline management
- async live integration coverage
- typed OOP and managed-lifetime tests
- optional PyO3 native wheels
- TestPyPI
- PyPI
- post-release verification
- self-hosted runner documentation

There is no glamour in this list, which is exactly why it is valuable.

A Brief Tour of the Public Surface

At the highest level, the package gives you:

- `GemStoneConfig`
- `GemStoneSession`
- `TransactionPolicy`
- `session_scope(...)`
- `PersistentRoot`
- `GSCollection`
- `GStore`
- `ObjectLog`
- `TypedOop[T]` and managed OOP handles
- `AsyncSession` and FastAPI helpers
- concurrency helpers
- Flask request-session providers
- optional native backend discovery
- benchmarks and release utilities

That list is broad, but it is not random.

It maps to the actual lifecycle of production software:

1. connect
2. do work
3. persist state
4. expose it through an app
5. survive contention
6. measure it
7. release it

That is a proper narrative arc. It is almost heroic. Only the benchmark JSON reports prevent it from becoming fantasy literature.

The Package Is Also a History Lesson

One subtle thing `gemstone-py` does well is acknowledge ancestry without trapping itself inside it.

There are older traditions around GemStone and MagLev-style workflows that influenced how people originally approached this space. The current package does not deny that history. It simply refuses to keep dragging unnecessary runtime assumptions into the present.

That is why the package now emphasizes:

- plain GemStone support
- explicit packaging
- maintained workflows
- current examples

You inherit the useful ideas without keeping the old scaffolding that no longer helps.

The First Joke You Should Internalize

Every ecosystem has a joke that secretly contains its survival guide.

In this one, the joke is:

> "The stone remembers everything, including your mistakes."

Funny? Mildly.

Useful? Extremely.

GemStone is persistent. That means:

- your writes matter
- your commits matter
- your names matter

- your migrations matter

The package will help you, but it will not infantilize the repository into a temporary playground. That is a feature.

What You Should Expect as a User

You should expect three things from `gemstone-py`.

First, clarity.

The package increasingly chooses explicit policy over convenience that only feels friendly until it corrupts a workflow.

Second, maintained examples.

The examples are not decorative garnish. They are part of the teaching surface.

Third, operational seriousness.

The package now publishes, verifies, benchmarks, compares, and checks itself in ways that many far larger projects postpone indefinitely.

So the promise is not "magic persistence."

The promise is:

> "A disciplined Python surface over GemStone, with enough tooling that you can trust what the package claims."

What You Should Not Expect

You should not expect:

- accidental auto-commit everywhere
- a fake ORM that hides repository semantics until the worst possible moment
- examples that teach one API while the package ships another
- benchmark claims without artifacts
- release claims without proof

The package has worked hard to stop doing those things.

The result is sometimes slightly less cuddly than a typical convenience library. It is also vastly easier to defend in front of other engineers.

Screenshot Intermission

One of the nicer signs of maturity in a package is when it can verify itself in a machine-readable way after installation.

`gemstone-py` has that.

That means users do not have to interpret success based on the emotional tone of one print statement and a half-finished shell script.

The Human Value of a Good Setup Story

This package now has a setup story, a manual, examples, a cookbook, benchmark policy, release policy, runner docs, and post-release verification. That sounds administrative until you have lived without it.

Without those things:

- onboarding is guesswork
- release day is improvisation
- benchmarks are mythology
- and the person who knows how it works becomes a bottleneck with a pulse

With those things:

- new users start faster
- maintainers sleep better
- the repository has fewer secrets

That is not just technical quality. That is social quality.

Why This Introduction Is So Long

You may reasonably ask why a package introduction needs to be this long.

Three reasons:

1. because GemStone is important enough to deserve a proper explanation
2. because production packages benefit from narrative documentation, not only reference pages
3. because the author of this book clearly believes that jokes and page breaks are cheaper than confusion

The third reason is, admittedly, difficult to refute.

End of Part I

You now know:

- why the package exists
- what problem it solves
- what sort of package it is trying to be
- what promises it actually makes

Next comes the real center of gravity: sessions and transactions.

That is where most users either become comfortable or become poets of regret.

Part I Notes Page

- Package goal: direct, disciplined GemStone access from Python
- Central virtues: explicit policy, maintained examples, release proof, benchmark proof
- Central warning: the repository is persistent and remembers your behaviour

If you remember only one line from this part, make it this one:

> Convenience is nice. Correct transaction boundaries are nicer.

Part II: Sessions and Transactions

Opening Claim

If Part I explained why the package exists, Part II explains whether it will behave when money, users, or deadlines are involved.

That behaviour lives in sessions and transactions.

This is where many persistence systems become vague. `gemstone-py` has instead become rather blunt, which is healthy.

What a Session Is

A `GemStoneSession` is a live connection to the stone through the GCI client library. It is not:

- a Python-only cache
- a decorative context manager
- a magical bundle of database vibes

It is the thing through which real repository work happens.

When you open a session, you are establishing a concrete relationship with the stone. That relationship includes:

- login state
- active transaction state
- visibility of committed and uncommitted work
- failure modes that are more honest than most web developers prefer before lunch

Configuration First, Not Last

Sessions are built from configuration, and that is one of the package's most important improvements over the "just pass strings around until something works" school of engineering.

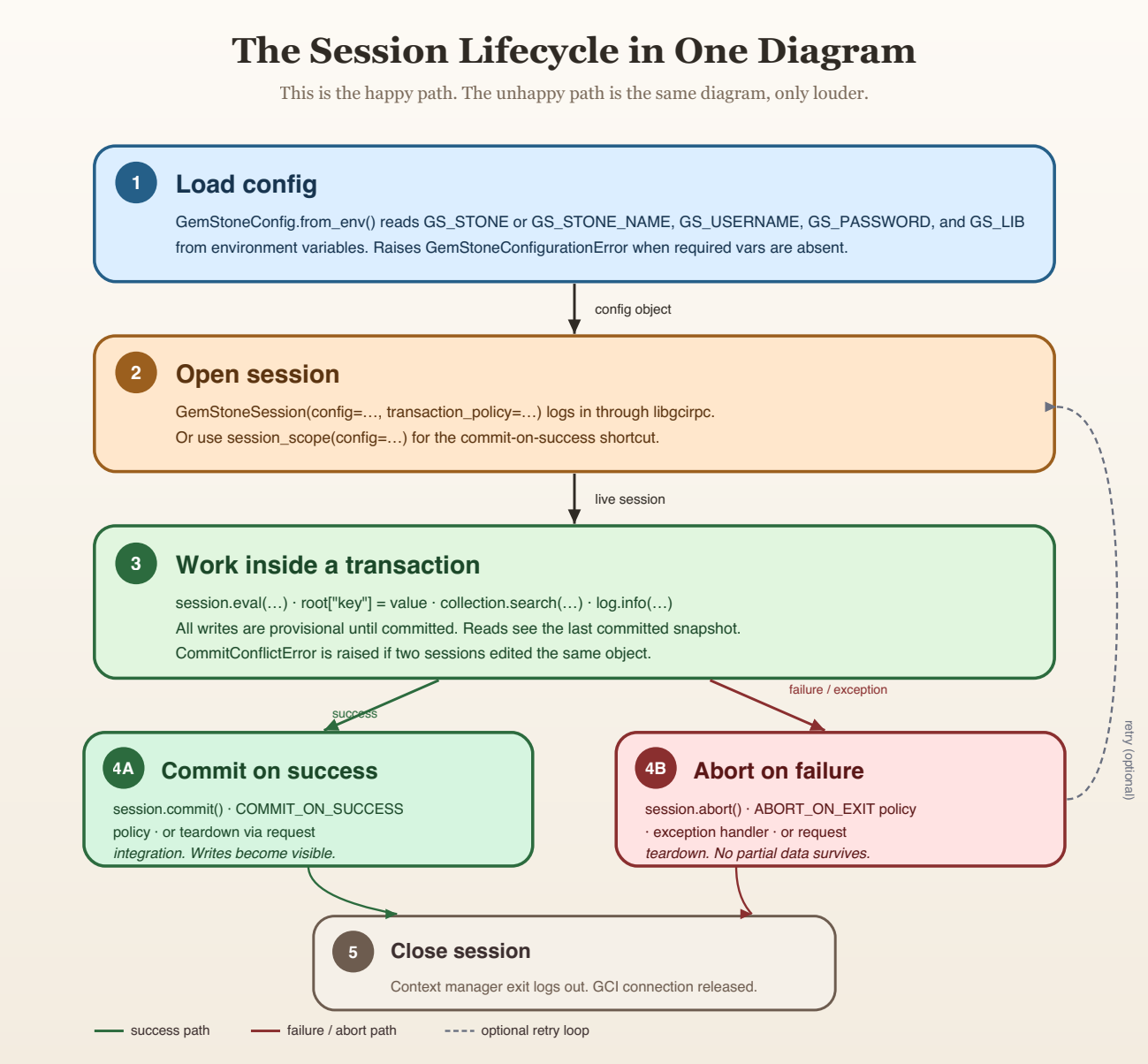
`GemStoneConfig.from_env()` is the normal entry point.

This is valuable because:

- credentials are validated explicitly
- host and stone settings are centralized
- the same config object can be shared by sessions, stores, benchmarks, and web providers

It also means your login story can be explained in one sentence instead of six fragments and a shrug.

The Session Lifecycle in One Diagram



What the diagram is telling you:

1. load configuration
2. open a session
3. do work in a transaction
4. either commit or abort

That is all.

That is also everything.

Many bad persistence stories happen because somebody mentally removed step 4 and replaced it with "the library will surely do what I meant." This package works hard to stop that sentence from succeeding.

Manual Is the Honest Default

One of the most consequential package decisions is that `GemStoneSession(...)` defaults to `TransactionPolicy.MANUAL`.

At first this sounds slightly severe. Then you remember what hidden auto-commit behaviour does to teams, and it starts to sound merciful.

Manual means:

- session exit does not imply commit
- writes are not made durable by accident
- low-level code does not hide mutation semantics

The price is that you must think. The reward is that your data model is not held together by misunderstood context manager etiquette.

The Three Policies

You should treat `TransactionPolicy` like traffic signage rather than flavor text.

`MANUAL`

Use when:

- you want direct control
- you are writing infrastructure or lower-level code
- you will call `commit()` or `abort()` yourself

`COMMIT_ON_SUCCESS`

Use when:

- the work in the block is one clear write unit
- success should publish the change
- exceptions should roll it back

`ABORT_ON_EXIT`

Use when:

- the work is read-only or exploratory
- you want a guaranteed clean exit

None of these are abstract philosophy. They are repository behaviour.

`session_scope(...)` Is the Friendly Path

If `GemStoneSession` is the direct instrument, `session_scope(...)` is the musically literate friend who reminds you what key the song is in.

It is usually the better choice for application code because it makes the unit of work obvious:

- enter scope
- do the work
- commit on success
- abort on failure

This pattern fits services and request handlers naturally. It also keeps commit semantics close to the application boundary, which is a polite way of saying "fewer people will accidentally commit from somewhere bizarre."

The Classic Mistake: "It Worked, But the Data Is Gone"

This sentence is so common it deserves a plaque.

What it usually means:

- you wrote data
- you saw it inside the current transaction
- you exited a manual session
- you never committed

The package did not betray you. The package did exactly what you told it to do.

The fix is not a mysterious GemStone ritual. The fix is one of:

- use `COMMIT_ON_SUCCESS`
- use `session_scope(...)`
- call `commit()` explicitly

A Relationship Analogy, Unfortunately Accurate

Sessions are like relationships. Transaction policies are like boundaries.

Manual:

> "We should be explicit about expectations."

Commit on success:

> "If this goes well, we are telling the world."

Abort on exit:

> "This was just coffee. No assumptions."

The analogy becomes less funny once you realize how many production bugs come from engineers who treat all three states as emotionally interchangeable.

Nested Transactions and Conflict Handling

The package includes nested transaction helpers because real applications do not remain single-layered for long.

Nested transactions are useful when:

- outer work should survive
- inner work may fail or conflict
- you want a smaller retryable unit inside a larger operation

That is not an exotic pattern. It is ordinary once your system has enough moving parts to deserve tests and a pager.

The important rule is to keep the retryable section as small and concrete as possible. Retrying half a business workflow is a confession, not a design.

Aborts Are Not Shameful

Many teams talk about aborts as though they were embarrassing accidents. In a transactional system, aborts are simply one of the valid outcomes.

You abort because:

- the work was exploratory
- an exception happened
- a conflict occurred

- the request ended in failure
- you are preserving a clean transactional story

An abort is often proof that the package is doing its job.

The shame would be half-committed state dressed up as success.

What Web Integration Changed

One of the meaningful hardening changes in the package was to make Flask request session finalization happen in teardown rather than pretending an `after_request` hook could safely decide the world.

That matters because handled `500` paths exist.

If the request lifecycle commits too early, you get a nasty class of bugs:

- the app returns an error
- the user sees failure
- but partial repository work still commits

That kind of bug is pure administrative sorrow. The package has already done the work to avoid it, which means users should actually use the provided web helpers.

Session Pools and Thread-Local Providers

As soon as you move into web applications, session reuse enters the chat.

The package offers:

- `GemStoneSessionPool`
- `GemStoneThreadLocalSessionProvider`

The pool is the more generally production-friendly option:

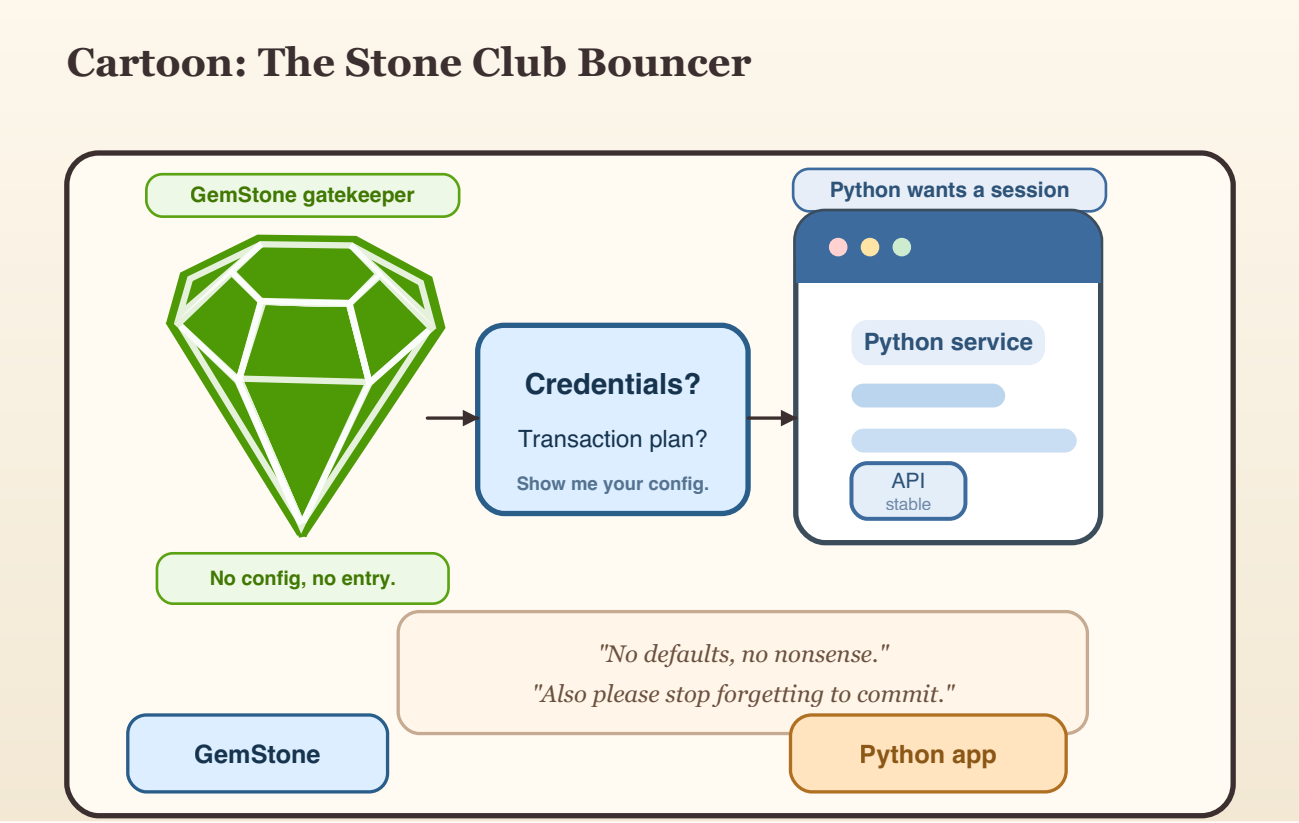
- bounded reuse
- explicit close and health tracking
- better operational visibility

The thread-local provider is simpler when your hosting model is stable and one session per thread is the clearest story.

Neither option changes the central truth:

the session still represents a real transactional relationship with the stone.

A Bouncer Cartoon Because You Have Earned It



This cartoon is silly, but the caption is not:

> "No defaults, no nonsense. Also please stop forgetting to commit."

That is approximately the package's approach to transaction design.

Practical Session Rules

If you want a short operational checklist:

1. build config once
2. use `session_scope(...)` for normal units of work
3. reserve raw manual sessions for infrastructure or scripts
4. commit only when success is clear
5. abort without shame
6. let request teardown own final web transaction state
7. assume conflicts will happen eventually

These rules are not glamorous. They are reliable.

A Tiny Worked Example

Here is the most important write pattern in the whole package:

```
from gemstone_py import GemStoneConfig, session_scope
from gemstone_py.persistent_root import PersistentRoot

config = GemStoneConfig.from_env()

with session_scope(config=config) as session:
    root = PersistentRoot(session)
    root["Users"] = {"count": 1}
```

This small example already tells a useful story:

- config is explicit
- the session is scoped
- commit behaviour is clear
- data goes to a named repository location

Many production features are just more ambitious versions of this sentence.

End of Part II

If you understand this part, you understand the package's moral center.

Next we move into the first major persistence abstraction: `PersistentRoot`.

That is where things become both friendlier and slightly more dangerous, because now the names you choose start to become part of the repository's public memory.

Part II Notes Page

- Session = real connection plus transaction state
- Default session policy = manual
- Best normal application path = `session_scope(...)`
- Aborts are healthy
- Request teardown owns final Flask transaction outcome

If you remember only one line from this part, make it this one:

> Explicit transaction policy is a kindness to your future self.

Part III: PersistentRoot and Friends

Opening Observation

When people first arrive at `PersistentRoot`, they often experience a pleasant moment of relief.

After sessions, policies, and login details, here at last is something comfortingly direct:

```
root["MyThing"] = {...}
```

Yes. Exactly. That is why people like it.

It is also why people can get themselves into trouble with it if they mistake directness for absence of consequence.

What `PersistentRoot` Really Is

`PersistentRoot(session)` is the friendly Python surface over GemStone-backed named repository dictionaries, especially `UserGlobals`.

The package also gives you:

- `PersistentRoot.globals(session)`
- `PersistentRoot.published(session)`
- `PersistentRoot.session_methods(session)`

This matters because GemStone already has the idea of named global dictionaries. The library is not inventing fake storage out of local Python air. It is giving you a manageable way to work with repository-native structures.

Why Users Love It Immediately

Three reasons:

1. It is easy to explain.
2. It is easy to inspect.
3. It makes persistent state feel legible.

There is enormous value in being able to say:

> "This application stores its top-level state under these named keys."

That sentence is both technical and social. It helps the code and the humans.

The Four Symbol Dictionaries

The package example tour intentionally shows all four major dictionary surfaces:

- `UserGlobals`
- `Globals`
- `Published`
- `SessionMethods`

For most application work, `UserGlobals` is the right home.

Why?

- it is the least surprising
- it suits application-owned names
- it maps naturally to "our app keeps its state here"

The other dictionaries matter, but they should not become your casual dumping ground just because they exist and seem adventurous.

The Difference Between a Root and a Junk Drawer

`PersistentRoot` works best when you impose naming discipline.

Good:

- `CustomerDirectory`
- `WorkQueue`
- `InvoiceCounters`
- `BlogPosts`

Bad:

- `Temp`
- `Stuff`
- `Data2`
- `ThingActuallyFinal`

Worse:

- `Temp2`
- `Temp3`
- `ActualTemp3Fixed`

Persistent names tend to outlive the emotional state in which they were created. Name things like somebody else will have to read them in six months, because somebody else will. Frequently that somebody else is you, but angrier.

Simple Structures Work Well

One reason `PersistentRoot` is such a good starting point is that simple Python structures map well to simple persistent structures:

- dictionaries
- strings
- numbers
- lists of basic values

That lets you build momentum fast.

The package later gives you more specialized helpers for:

- indexed collections
- store-like access
- append-only logs
- concurrency primitives

But you do not need all of those on day one. `PersistentRoot` lets you start with state that is named, readable, and durable.

The Grand Tour Example Makes This Concrete

The main `examples/example.py` file shows:

- reading `UserGlobals`
- inspecting keys
- writing named dictionaries
- confirming cross-session visibility
- proving a second session sees committed work
- proving an abort refreshes same-session state

That sequence matters because it turns persistence from a vague concept into a demonstrated behavioural fact.

If a package claims persistence, it should show it across sessions. This one does.

Cross-Session Visibility Is the First Real Magic Trick

Consider the sequence:

1. session A writes a dictionary
2. session A commits
3. session B opens independently
4. session B reads the dictionary

That is the first moment many users stop thinking "Python wrapper" and start thinking "repository-backed system."

The distinction matters. You are not merely serializing data in the background. You are working with shared durable state that outlives the original process.

When `PersistentRoot` Is Not Enough

You should move beyond `PersistentRoot` when:

- search patterns matter
- indexing matters
- the dataset is large enough that key-by-key navigation becomes clumsy
- your access pattern wants a store abstraction
- your system benefits from event logs or concurrency helpers

This is not a failure of `PersistentRoot`.

It is success with enough growth that you now deserve specialization.

Friends of `PersistentRoot`

The useful "friends" in the title are:

- `GemStoneSessionFacade`
- `GSCollection`
- `GStore`
- `ObjectLog`
- `RCCounter`
- `RCHash`
- `RCQueue`

They are "friends" because they often sit under a named root key or alongside root-managed application state.

Examples:

- store a `RCQueue` as `root["WorkQueue"]`
- store a named collection and use it through `GSCollection`
- use root keys to organize top-level domain buckets

A Good Application Shape

A sensible pattern for many apps is:

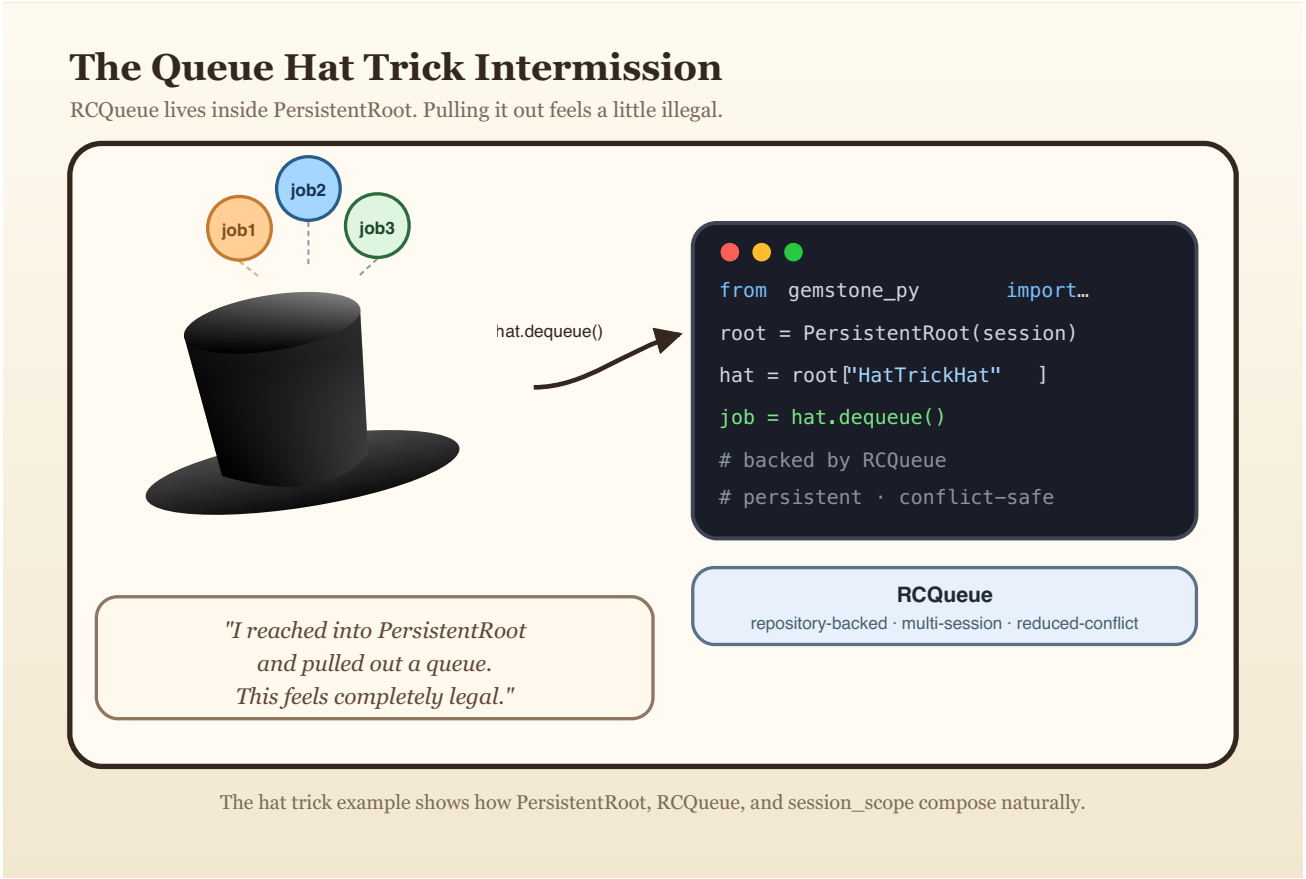
- `PersistentRoot` owns top-level names
- specialized helpers own the inner behaviour

For example:

- `root["UsersConfig"]` -> simple dict
- `root["WorkQueue"]` -> `RCQueue`
- `GSCollection("Accounts")` -> indexed account records
- `ObjectLog(...)` -> audit events

That balance keeps the repository understandable at the top and specialized where it needs to be.

The Queue Hat Trick Intermission



The hat trick example shows how PersistentRoot, RCQueue, and session_scope compose naturally.

The hat trick example is funny because it takes a real shared queue and gives it stage props.

That joke contains a good lesson:

you can use playful examples to teach serious repository behaviour, as long as the underlying abstraction is real.

The queue is real. The hat is branding.

Batched Reads Matter More Than They First Appear

One of the quieter improvements in the package is that `PersistentRoot` mapping operations like `keys()`, `items()`, and `values()` have been optimized to use batched repository fetches rather than unnecessarily chatty per-entry patterns.

That matters because persistence APIs are not only judged by correctness. They are also judged by whether they behave like they respect the user's time.

A helper that is simple but slow on normal operations becomes difficult to love.

The package has put real effort into preventing that.

Naming Strategies That Age Well

Good naming strategies for root keys:

- noun phrases for durable collections: `CustomerDirectory`
- action-neutral queues: `OutboundEmailQueue`
- counters that describe their meaning: `InvoiceSequence`
- logs named by domain: `BillingAuditLog`

Less good strategies:

- temporary emotional state
- migration history disguised as final names
- abbreviations only one person understands

If the key will probably survive longer than your current coffee, give it a name worthy of survival.

A Useful First Refactor

Many early apps start with one large root dictionary. That is acceptable up to a point. The first useful refactor is usually to split that single blob into:

- domain buckets
- specialized helpers

- clearly named operational structures

That keeps inspection pleasant and reduces the tendency for everything to become a giant dictionary with unexplained interior politics.

End of Part III

At this point you know why `PersistentRoot` is the easiest serious place to begin. It is direct, durable, and visible.

Next we move into the helpers you reach for when you need more structure:

- indexed querying
- store-shaped persistence
- persistent event logging

That is where the package starts to feel less like a bridge and more like a small ecosystem.

Part III Notes Page

- `PersistentRoot` is the simplest persistent entry point
- `UserGlobals` is usually the right home
- naming discipline matters more than people expect
- simple persistent state is a feature, not a beginner trap
- specialization begins when search, indexing, or shared primitives justify it

If you remember only one line from this part, make it this one:

> Use `PersistentRoot` as a well-named front door, not as a junk closet.

Part IV: Queries, Stores, and Logs

Opening Sentence

At some point every application graduates from "I have a persistent dictionary" to "I need a shape for repeated lookup, history, or search."

That is when the specialist tools step onto the stage:

- `GSCollection`
- `GStore`
- `ObjectLog`

Each one solves a different problem. The trick is not merely knowing that they exist. The trick is knowing when each one is the least surprising tool.

`GSCollection`: When Search Begins to Matter

`GSCollection` is for collections you want to query by stored attributes.

That makes it the natural next step after `PersistentRoot` when you discover that "top-level key plus nested dict" is no longer enough.

It is good for:

- named logical collections
- indexed lookups
- search operations repeated often enough to deserve respect

It is not good for:

- replacing all dictionaries just because indexing sounds sophisticated
- pretending you now have an ORM universe and therefore wisdom

A Good Reason to Use `GSCollection`

You have records like:

```
{"name": "Ada", "city": "London", "team": "Platform"}
```

And you need to do this repeatedly:

- find everyone in London
- find all records for a given team
- keep the collection named and reusable

That is `GSCollection` territory.

The package examples around indexing do a good job of teaching exactly this moment, which is why they remain worth reading even if you already know the API.

Indexes Are a Promise to Your Future Self

Creating an index is not a performance charm or a badge. It is a promise:

> "We expect to look this up often enough that we care."

That promise helps in two ways.

First, it speeds the workload.

Second, it clarifies intent in the codebase. Somebody reading the collection definition learns which queries the system actually values.

That is an underrated form of documentation. Good indexes explain the workload.

`GStore`: A Repository Store That Feels Like a Store

`GStore` exists because many programmers want a store-shaped interface even when the underlying persistence is a GemStone repository.

This is a reasonable desire.

Sometimes the best application abstraction is:

- string-ish key
- structured value
- named store
- direct read/write semantics

That is exactly the space `GStore` occupies.

Why `GStore` Is Nice

It is easy to explain:

```
store["sku:hat"] = {"name": "Hat", "stock": 8}
```

It is also honest.

The values are still persisted through GemStone, but the programming model is store-oriented rather than collection-oriented.

That distinction is subtle and useful:

- `GSCollection` is about record collections and queries
- `GStore` is about named key/value persistence

Confusing the two makes APIs worse and conversations longer.

`ObjectLog`: When the Past Needs a Filing System

`ObjectLog` is for events that should live in the repository with the data they describe.

This is not the same thing as standard Python logging.

Python logging answers:

> "What did the process say at runtime?"

`ObjectLog` answers:

> "What durable event record should remain attached to this system?"

That difference matters when audits, history, and postmortems become real.

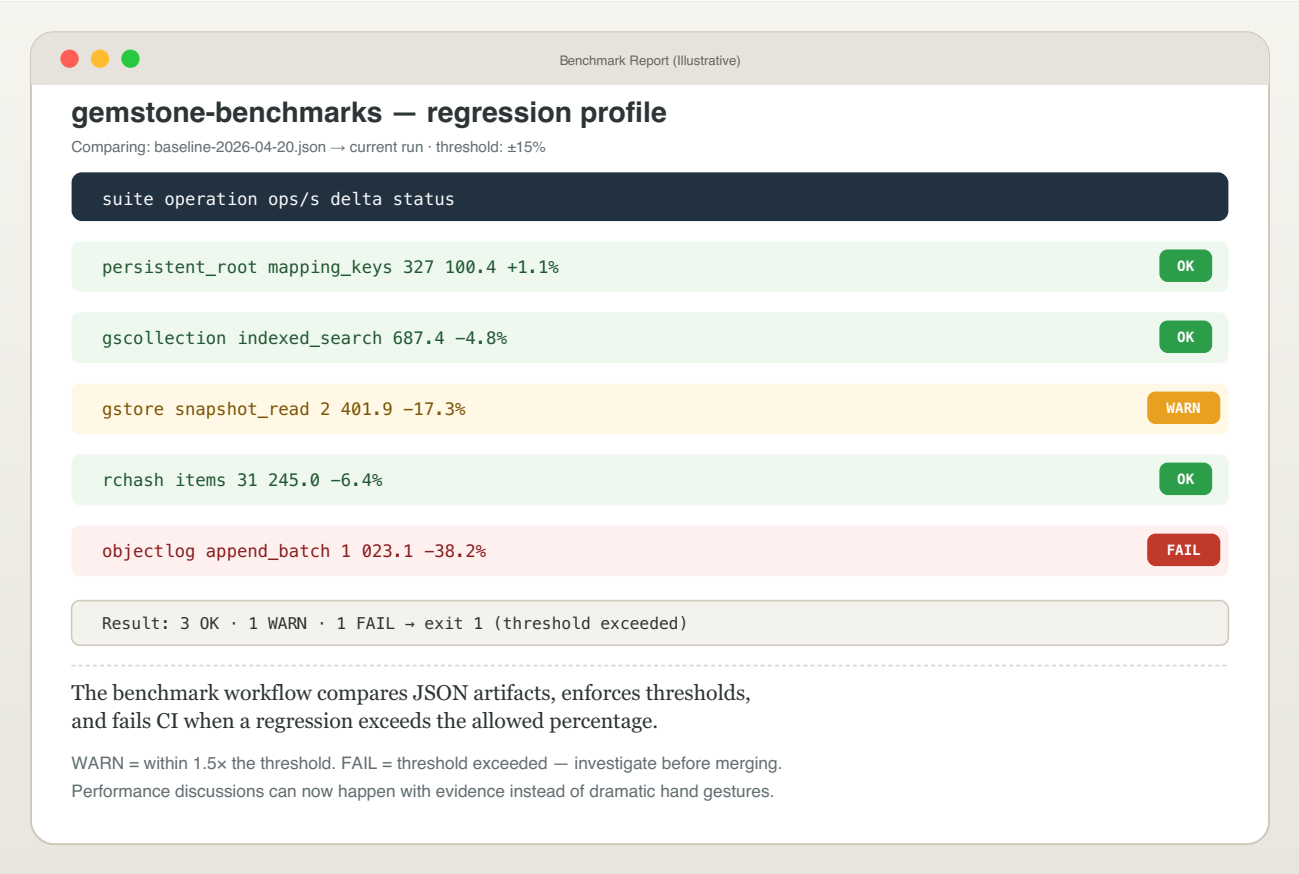
The Psychological Value of a Good Log

Systems without durable event logs create a familiar species of meeting:

- several people are present
- nobody knows exactly what happened
- everyone uses verbs like "probably" and "maybe"
- one person says "I think it was after deploy"
- another says "which deploy"

An `ObjectLog` does not solve every mystery, but it improves the quality of your ignorance. That is a large operational gift.

Screenshot Intermission: Benchmarks Are Not the Same as Examples



The package wisely separates examples from maintained benchmark tooling.

That separation is part of the same design discipline that makes the persistence helpers useful. Teaching code is not measurement policy. Measurement policy is measurement policy.

Choosing Between the Three

Use `PersistentRoot` when:

- you want named top-level state
- lookups are direct

Use `GSCollection` when:

- indexed search matters
- records behave like a queryable collection

Use `GStore` when:

- a key/value store model is the clearest fit

Use `ObjectLog` when:

- the history itself is a first-class output

The point is not to pick a winner. The point is to stop forcing one abstraction to cosplay as all the others.

Performance Work Happened Here Too

These helpers are not just API wrappers. They have received real batching and round-trip reduction work.

Why mention that in a user book?

Because the shape of the abstraction affects how much users trust it.

A helper that reads as though it respects the repository but then performs like it resents the network is difficult to recommend with a straight face.

The package now has:

- batched mapping fetches
- benchmark reports
- comparison tooling
- environment-specific baselines

That makes the performance story concrete instead of ceremonial.

The Main Example Ties Them Together

The large `examples/example.py` script shows the specialist helpers not as isolated toys but as neighbours in one coherent package:

- root for top-level names
- store for key/value persistence
- object log for events
- concurrency helpers for shared mutable state

This is important.

A library becomes easier to trust when the major pieces feel like they were designed to live in the same house, rather than as distant cousins introduced at a wedding.

A Joke About Naming Stores

If your `GStore` files are named:

- `db.db`
- `new.db`
- `test2.db`

then congratulations: you have re-created the personality of a temporary directory inside a persistent system.

Name them like they are real.

Your benchmark reports, cleanup scripts, and future self will all appreciate it.

A Practical Design Heuristic

When in doubt, ask:

> "What question will I ask this data most often?"

If the answer is:

- "What is the top-level thing named X?" -> `PersistentRoot`
- "What value is stored at key Y?" -> `GStore`
- "Which rows match this attribute?" -> `GSCollection`
- "What happened over time?" -> `ObjectLog`

That question is often enough to choose correctly.

End of Part IV

You now have the package's major persistence helpers in view.

Next we move to the place where people become either very happy or very haunted:

the web layer.

Request lifecycles make transaction design visible in a way scripts rarely do. That is why the next part exists.

Part IV Notes Page

- `GSCollection` is for indexed record-style lookup
- `GStore` is for key/value persistence

- `ObjectLog` is for durable event history
- do not force one abstraction to do all jobs badly
- the package backs the abstractions with real benchmark and batching work

If you remember only one line from this part, make it this one:

> Choose the helper by access pattern, not by mood.

Part V: Web Apps and Request Lifecycles

Opening Confession

A persistence package can feel excellent in scripts and still behave terribly in web applications.

Why?

Because request lifecycles are where hidden transaction assumptions go to become user-visible incidents.

This part exists so you do not have to learn that lesson exclusively through production embarrassment.

The First Rule of Web Persistence

A request is not merely a function call with fashionable headers.

It is a unit of work with:

- inputs
- application logic
- error handling
- response generation
- and a final decision about whether repository changes should survive

That final decision is the heart of the matter.

If the persistence layer commits too early, you can easily wind up with:

- a failed request
- an unhappy user
- and a successful partial commit

That combination is offensively educational.

What the Package Provides

`gemstone_py.web` still gives you the compatibility surface:

- `install_flask_request_session(...)`
- `GemStoneSessionPool`
- `GemStoneThreadLocalSessionProvider`
- `session_scope(...)`
- provider snapshots
- close hooks
- request lifecycle wiring

The reusable providers now live in `gemstone_py.session_providers`, while Flask-specific imports also exist under `gemstone_py.frameworks.flask`.

This is not a random utility pile. It is the package's answer to the question:

> "How should a Python web app use GemStone responsibly?"

For async web stacks, `gemstone_py.aio.fastapi` and `gemstone_py.aio.litestar` give you:

- `AsyncSession`
- `session_dependency(...)`
- `GemStoneSessionMiddleware`

That is the package's answer to the adjacent question:

> "How should an async Python web app use GemStone without blocking the event loop directly?"

The Key Design Decision: Teardown Owns the Outcome

One of the most valuable hardening changes in the package was moving final request persistence decisions into teardown rather than letting an earlier hook pretend success had already been decided.

This matters for handled error paths.

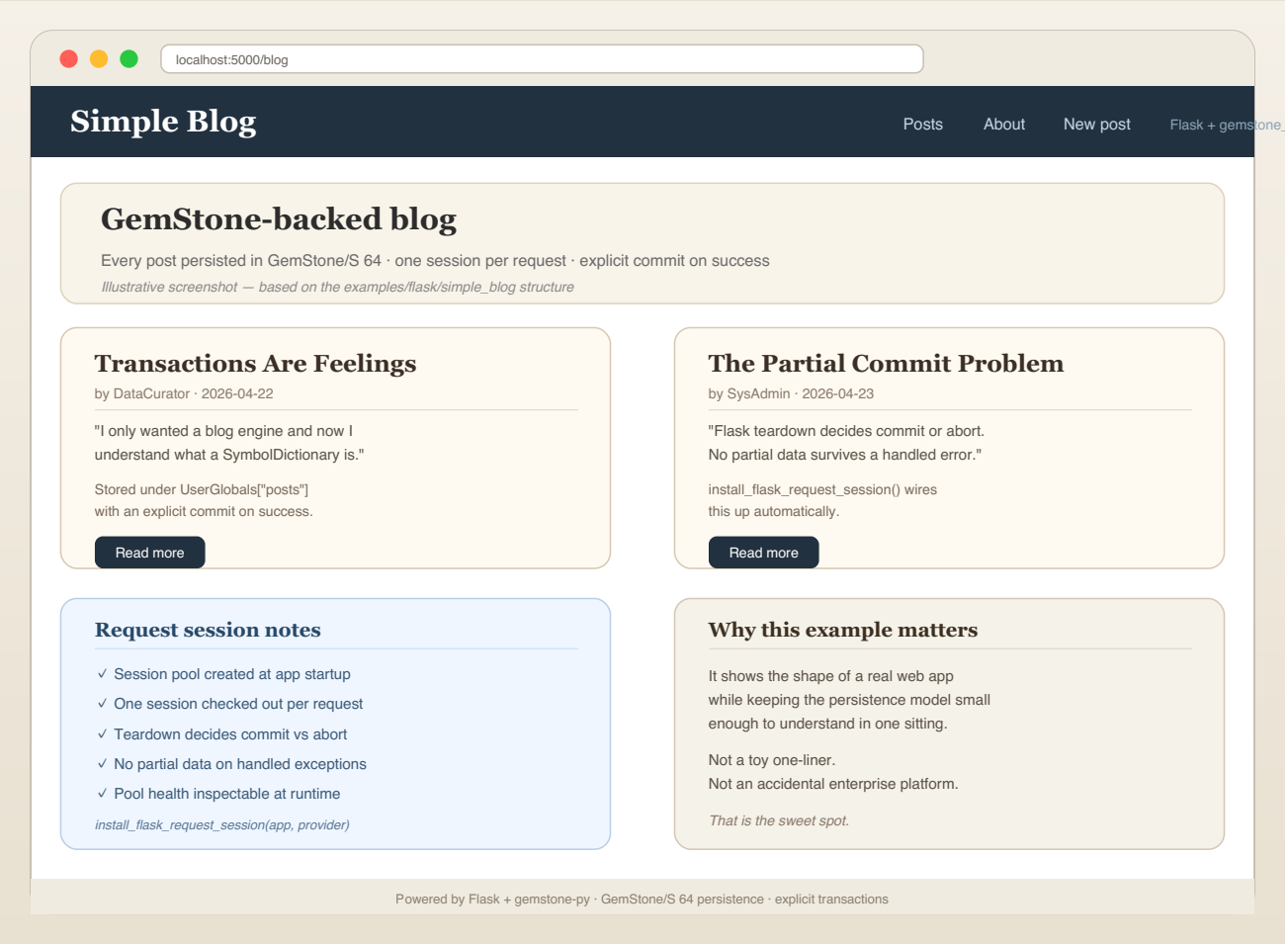
If the app catches an exception, returns a `500`, and the persistence layer has already committed, you have created the worst kind of bug:

- easy to trigger
- hard to explain

- and deeply annoying to users

The package now avoids that class of failure by design.

A Screenshot of the Kind of App This Enables



You do not need a huge app to benefit from request-scoped session handling.

Even a small Flask application with:

- a few routes
- a persistent model
- and some forms

already benefits from predictable commit-or-abort semantics.

Pools Versus Thread-Local Providers

You have two main provider styles.

`GemStoneSessionPool`

Use this when:

- concurrency is real
- request workers are multiple
- you want bounded reuse and operational accounting

`GemStoneThreadLocalSessionProvider`

Use this when:

- one session per thread is the clearest mapping
- the threading model is simple and stable

Do not choose based on whichever name sounds more advanced. Choose based on actual runtime shape.

Why Operational Visibility Matters

The provider layer has snapshots, health checks, counters, and lifecycle hooks because web session reuse is not merely a programming abstraction. It is an operational system.

That means somebody will eventually ask:

- how many sessions are in use?
- are they healthy?
- are they being recycled?
- did a request leak one?

If the package cannot answer those questions, the application will eventually answer them by failing in public.

The `simple_blog` Example

The `examples/flask/simple_blog/` app is valuable because it is modest.

That sounds strange, but modesty is an underrated teaching property.

The example is large enough to demonstrate:

- routes
- persistent state
- request integration

But small enough that a user can still keep the application shape in their head.

That makes it a much better first web example than a sprawling demo with seven screens and an identity crisis.

The `magtag` Example

Then there is `magtag`, which is the package saying:

> "Fine. You want a larger app. Here is a larger app."

It demonstrates:

- more routes
- more state
- collection-backed behaviour
- more realistic interaction flow

It is useful not because you will copy it verbatim, but because it proves the package can live inside a web application with enough surface area to trigger real integration concerns.

Session Middleware Is a Moral Choice

When a web package decides where and how persistence happens during requests, it is making a moral choice about who gets surprised later.

Good middleware surprises fewer people.

Bad middleware says:

- "I committed already"
- "I assumed the response meant success"
- "I hope your error path was decorative"

The current `gemstone-py` web design is good specifically because it stopped doing that.

Production Guidance, Short Version

If you are using Flask:

1. install request-session handling explicitly
2. prefer a pool unless you have a clear reason not to
3. keep request logic inside the request/session lifecycle
4. let teardown decide final commit vs abort
5. expose provider snapshot or health data in admin or ops endpoints

This is not over-engineering. It is the difference between a system you can debug and a system you can narrate only in retrospect.

If you are using FastAPI:

1. use `gemstone_py.aio.AsyncSession`
2. install `session_dependency(...)` for request-scoped sessions
3. wrap writes in `async with session.transaction()`
4. keep blocking GemStone calls inside the async facade
5. run `examples/fastapi/app.py` before adapting it to a larger app

The "Handled 500" Story

One of the package's own live workflow failures helped prove the value of the current design.

A handled `500` path exposed that earlier lifecycle logic could still commit work before teardown had the chance to abort it properly.

That failure was useful because:

- it happened in a real workflow
- it had a concrete symptom
- it led to a better design

The right lesson is not "how embarrassing." The right lesson is "good packages allow their own verification lanes to teach them something important."

Web Development Is Where Discipline Pays Off

In scripts, sloppy transaction semantics may sleep quietly for a while.

In web apps:

- users race each other
- handlers fail halfway through
- retries happen
- timeouts happen
- pool state matters

This is why the package's investment in session providers, live tests, soak tests, and request teardown logic is not optional garnish. It is the core of trustworthy web use.

A Joke About Flask Extensions

Many Flask extensions silently assume they are the only adults in the room.

`gemstone-py` has taken the opposite attitude:

> "The request lifecycle is shared property, so our transaction story had better be explicit and testable."

That is less romantic than black-box convenience, but much better if you enjoy applications that can survive contact with errors.

End of Part V

Web integration is where the package proves it is not only a scripting bridge.

Next comes the part that frightens everybody just enough to be useful:

concurrency, conflicts, shared state, retries, and the rules for not becoming a folktale told by more careful engineers.

Part V Notes Page

- request lifecycles are real transaction boundaries
- teardown should own final commit vs abort
- pools are generally the safer production default
- thread-local providers are good for simpler hosting models
- live and soak tests matter here because theory is not enough

If you remember only one line from this part, make it this one:

> A failed request must not quietly become a successful commit.

Part VI: Concurrency, Conflicts, and Retries

Opening Warning

This part is where the package stops smiling politely and starts asking whether you actually mean what you say about shared state.

GemStone is not embarrassed by concurrency. It assumes you will need it.

The package therefore gives you real shared primitives, conflict visibility, and retry patterns. It does not promise a universe where everyone edits the same row and friendship magically wins.

Shared State Is the Point, Not an Accident

The concurrency helpers exist because some state really does belong in the repository rather than in one Python worker process.

That includes:

- counters
- queues
- shared hashes

When those structures matter across sessions, pretending they are local Python objects with a distant data layer is a design mistake. The package chooses to be honest instead.

The Core Helpers

The main actors are:

- `RCCounter`
- `RCHash`
- `RCQueue`

They are useful because they let multiple sessions coordinate through repository backed primitives.

They are dangerous because they let multiple sessions coordinate through repository backed primitives.

Those are the same sentence wearing different shoes.

`RCCounter`

Use `RCCounter` when the number must be shared and durable.

Examples:

- sequence generation
- shared progress counters
- coordination statistics

Do not use it when:

- a Python integer in one process would do
- the count is temporary and local
- you simply enjoy the phrase "repository-backed counter"

Taste matters.

RCHash

`RCHash` is the package's shared hash structure. It became especially interesting after batching improvements reduced the amount of chatter required to read its contents.

This is a useful reminder that concurrency helpers are not merely conceptual objects. They also need to perform.

A shared data structure that is correct but painfully chatty becomes a tax on every user. The package has put real work into making the hot paths more sane.

RCQueue

Queues are where concurrency suddenly stops being academic.

A queue immediately invites questions like:

- who adds work?
- who consumes work?
- what happens if two consumers try?
- how do we recover after partial processing?

These are good questions. They are the questions that make real systems interesting and occasionally noisy.

Commit Conflicts Are Not Personal Attacks

Let us say this clearly:

`CommitConflictError` is not a moral judgment.

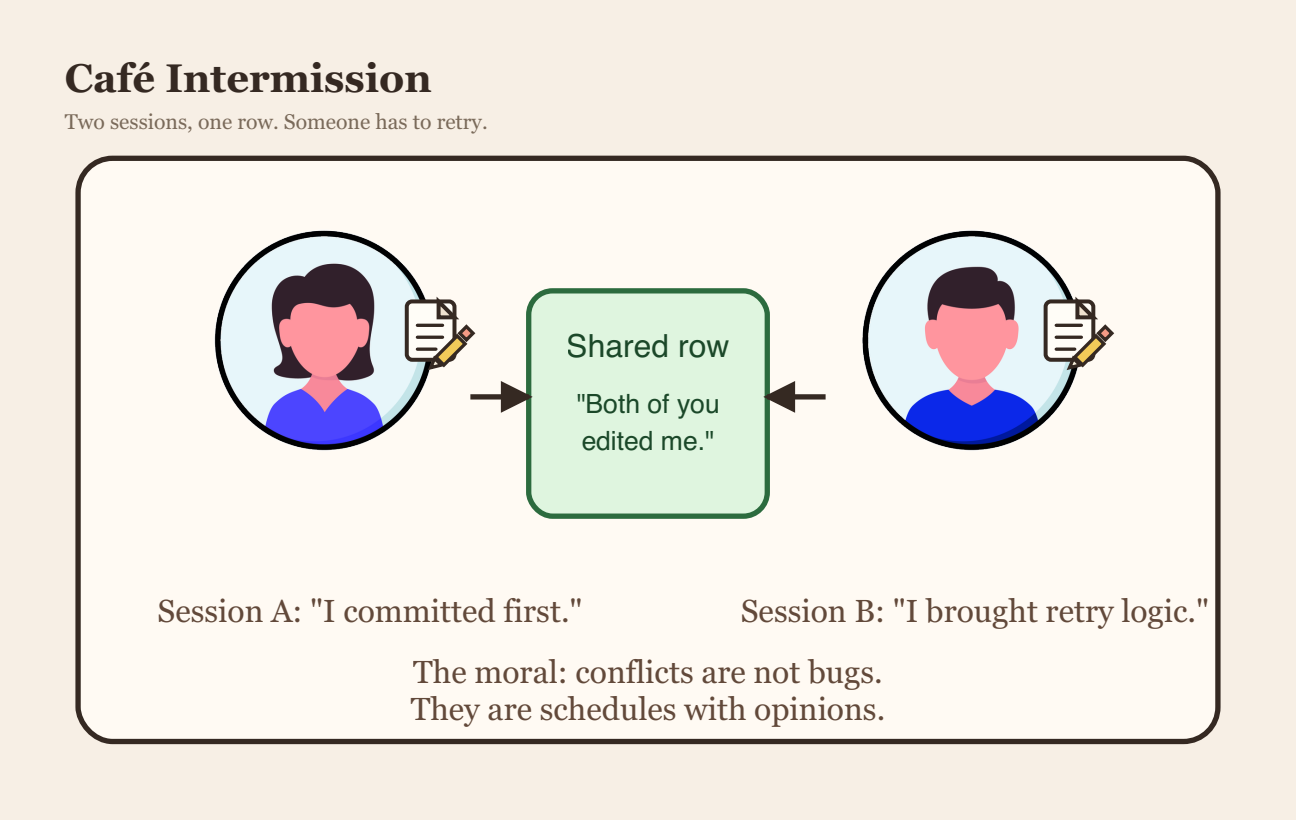
It means:

- two sessions touched overlapping state
- one of them committed first
- the other must retry, abort, or restructure its unit of work

That is all.

Treating conflicts as shocking betrayals is like treating a traffic light as a personal insult. The system is only telling you that other actors exist.

Cafe Intermission



This is the whole concept in one image:

- two sessions
- one shared row
- one clean explanation for why only one of them gets to pretend it was first

The joke works because the underlying logic is exact.

Retry Logic Is Part of Design

Once a system has true multi-session writes, retries stop being an advanced technique and become part of normal design.

The right retry shape is:

- small unit of work
- explicit conflict boundary
- limited retry scope
- clear logging or visibility

The wrong retry shape is:

- "catch everything"

- rerun half the application
- hope the world has changed enough to make this count as architecture

Locks Are Tools, Not Decorations

The package also exposes lock-related behaviour and instance inspection paths.

Locks are useful when:

- coordination needs stronger guarantees
- shared access patterns are explicit
- contention must be managed rather than merely observed

They are not useful when added as ceremonial hardware to make a design sound serious in conversation.

Use them because the access pattern requires them, not because "enterprise" started whispering in your ear.

The Live Tests Matter Here More Than Almost Anywhere

Unit tests can teach API behaviour, but concurrency becomes genuinely credible only when live tests exercise:

- multiple sessions
- real conflicts
- real retry loops
- real pool and provider reuse
- lock-heavy paths

The package now includes repeated live contention and soak coverage because this domain refuses to be adequately explained by clever mocking alone.

Why Soak Tests Exist

A short contention test proves basic correctness.

A soak test is trying to answer a harder question:

> "Does this continue to behave under repeated reuse, pressure, and state recycling?"

That is exactly the kind of question that matters in long-running applications.

It is also exactly the kind of question that tends to go unasked until a system is old enough to have opinions and young enough to still page you.

Naming Shared Structures

If a queue, hash, or counter lives at the top level, name it like it will be inspected during an incident.

Good:

- `InboundInvoiceQueue`
- `ImportRetryCounter`
- `SessionCoordinationHash`

Bad:

- `TmpQueue`
- `Counter2`
- `MysteryHash`

The repository is not merely where data lives. It is where future explanations must begin.

A Practical Conflict Strategy

If you expect conflicts:

1. isolate the shared state
2. reduce the transaction surface
3. log enough context to understand retries
4. keep the retry block narrow
5. accept that some paths must abort cleanly

This is how you keep concurrency from becoming a superstition.

The Secret Benefit of Good Conflict Handling

Good conflict handling makes teams calmer.

Why?

Because the system stops behaving like a haunted house. A conflict becomes:

- visible
- reproducible
- retryable
- documentable

That is much easier to live with than a system that "sometimes loses updates" and therefore forces everyone into dramatic oral history.

End of Part VI

You have now seen the package at its most honest:

- shared state
- conflicts
- retries
- soak behaviour

That leaves one final territory:

the boring, glorious machinery that proves a package can survive real life:

- benchmarks
- release workflows
- runners
- PyPI
- post-release verification

Part VI Notes Page

- shared state is real state
- conflicts are expected, not scandalous
- retries should be narrow and deliberate
- locks exist to solve patterns, not to impress people
- live contention and soak tests matter because concurrency lies in small demos

If you remember only one line from this part, make it this one:

> A conflict is a scheduling fact, not a character assessment.

Part VII: Benchmarks, Releases, and Operator Survival

Opening Thesis

A package becomes mature not when its API looks pretty, but when it can prove its claims under packaging, release, and operational pressure.

This part is about that proof.

The good news is that `gemstone-py` now has a real story here.

The bad news is that this story involves workflows, baselines, trusted publishers, runner service management, and PyPI metadata. In other words, it is the sort of material that turns junior engineers into maintainers.

Benchmarks Are Not Vibes

The package has a maintained benchmark lane. That matters because persistence helpers do not become good merely by being correct. They also have to avoid turning ordinary access patterns into performance tax forms.

The benchmark system now includes:

- CLI execution
- JSON artifacts
- report comparison
- baseline manifests
- environment-aware baseline selection
- threshold enforcement in GitHub Actions

That is what it looks like when performance work becomes policy rather than lore.

Smoke Versus Regression

One useful improvement in the benchmark workflow is the split between smoke and regression profiles.

This is a civilized solution to a common problem.

You often want:

- one cheap signal that the lane still runs at all
- one stricter signal for performance drift

Trying to force one benchmark policy to do both jobs usually produces:

- noisy failures
- unclear expectations
- and arguments about whether the benchmark "really counts"

Named profiles are cleaner.

Baselines Are Managed, Not Worshipped

The package can register benchmark baselines, compare reports against them, and select environment-matching baselines through a manifest.

That is important because one global baseline file is easy to describe and almost immediately too naive for reality.

Benchmarks are shaped by:

- runner type
- platform
- Python version
- stone configuration
- workload parameters

The tooling now acknowledges that. Reality is rude, but the tooling no longer pretends otherwise.

Releases Must Be Proven Twice

One of the package's strongest current features is its release discipline.

The story now includes:

- release dry run
- TestPyPI publish
- PyPI publish
- post-release verification against real PyPI
- installed-artifact contract checks

This is exactly how releases should work:

1. rehearse
2. publish to a rehearsal space

3. publish for real
4. verify the result as a user would see it

That is not overkill. That is the difference between "released" and "uploaded."

Trusted Publishing Was Worth It

The move to trusted publishing means release workflows no longer depend on long-lived API tokens passed around like cursed treasure.

Instead, the workflow identity itself is part of the release trust model.

That is cleaner and safer.

It is also slightly more bureaucratic the first time you set it up, which is how you know it is probably doing something useful.

The Post-Release Verify Workflow Is a Love Letter to Reality

The post-release verification workflow does something beautifully simple:

- waits for the version to appear on PyPI
- installs the published artifact
- runs package contract checks
- validates public CLI behaviour
- verifies metadata and long-description hygiene

This closes a painful gap that many packages leave open forever:

> "Yes, we published it. But did the thing the world receives still behave like the thing we intended?"

Now the answer can be verified automatically.

The Self-Hosted Runner Story

The live and benchmark workflows depend on a self-hosted runner because the stone and client library environment are real, stateful, and not meaningfully reproduced by a generic GitHub-hosted machine.

That means runner operations are part of package quality.

The package now includes:

- bootstrap scripts
- service installation scripts
- launchd integration
- health checks
- upgrade paths
- documentation

This is not glamorous, but it is adult.

Native Wheels Are Packaging Work, Not Just Speed Work

The optional `gemstone-py[fast]` install path adds a PyO3 native GCI extension. That sounds like a performance feature, and it is, but it also expands the release contract.

The package now has to prove:

- the pure ctypes fallback still works
- the native backend can be selected when installed
- Linux, macOS, and Windows wheels carry the expected tags
- ARM and x86_64 platforms are covered
- post-release verification installs the `fast` extra from real PyPI

That is why the native wheel workflow, post-release verify workflow, and backend examples belong in the same operational story as benchmarks.

A Runner Is Infrastructure, Not a Pet

There is a dangerous tendency to treat one carefully tuned self-hosted runner as a magical animal that simply exists and should not be questioned.

This is unwise.

A runner should have:

- labels
- a service definition
- update procedures
- health checks
- a failover or recovery story

In short, it should be infrastructure rather than folklore.

Release Notes Matter Too

The package now maintains:

- a changelog
- release metadata checks
- version/tag validation

This sounds bureaucratic until the first moment you need to answer:

> "What changed between 0.2.0 and 0.2.2, and why did we publish 0.2.1 at all?"

At that moment, you either have release discipline or you have interpretive dance.

Public Metadata Is Part of the Product

The work on PyPI long descriptions, project URLs, and metadata hygiene may sound cosmetic. It is not.

For many users, the PyPI page is the first product surface.

If that page contains:

- broken local absolute paths
- unclear URLs
- stale workflow references

then the package looks unmaintained before the user even installs it.

Metadata is not decoration. It is the front door.

The Release That Taught a Lesson

One useful episode in this package's history involved publishing a correct release and then noticing that the long description still contained local absolute paths.

The fix was not dramatic:

- clean the docs source
- cut another release
- verify the public metadata

But the lesson was excellent:

post-release verification is not vanity. It catches what normal build success cannot see.

A Joke About Maintainers

There comes a point in every package where the maintainer stops saying "wouldn't it be nice if this worked" and starts saying:

> "What exact workflow, artifact, release, and post-release verification proves that it worked?"

That is how maintainers are made.

Somewhere between the first trusted publisher entry and the first benchmark baseline manifest, a person quietly becomes more responsible than they intended.

The Full Lifecycle, Condensed

Here is the modern package lifecycle in one list:

1. code the change
2. run unit and live checks
3. benchmark if relevant
4. dry-run the release
5. publish to TestPyPI
6. publish to PyPI
7. verify the real release from PyPI
8. keep the runner healthy enough to do it again

That is a proper supply chain story, not a triumphant shell alias.

Final Cartoon-Free Advice

If you maintain this package or one like it:

- prefer boring workflow clarity over cleverness
- keep async, typed, and native examples aligned with the implemented API
- write docs for the administrator as well as the user
- keep benchmark policy evidence-based
- verify the thing the outside world actually installs
- treat self-hosted runners like infrastructure

These habits are not glamorous, but they compound into trust.

End of Part VII

You have reached the end of the book.

If you made it this far, you now understand `gemstone-py` from:

- connection setup
- session policy
- persistence design
- query and store helpers
- web integration
- concurrency
- benchmarks
- releases
- operations

That is a substantial amount of ground.

It is also enough ground to be genuinely useful.

Final Notes Page

If you remember only one line from the entire book, make it this one:

> Good persistence software is not just about writing data. It is about making the full lifecycle of that data understandable, testable, and survivable.

And if you remember only one joke:

> The stone remembers everything, including your mistakes.

That line has done a remarkable amount of educational work.

This PDF was generated locally from the Markdown sources in `docs/`. If you edit the source files, rerun `python docs/build_pdf_docs.py`.